Training loop

Từ batch tới checkpoint chạy lại được

Đóng notebook rồi sao?

Loss đã giảm nhưng mean/std thất lạc.

Model mới khởi tạo, kết quả đổi.

Ta cần cả pipeline:

dữ liệu → train → validation → lưu → suy luận mới.

01

Dữ liệu và một epoch

Dữ liệu có quy tắc biết trước

Nhãn 1 nếu .

72 mẫu train, 24 validation.

Fit mean/std trên train.

KẾT QUẢ

```
72 24 5
(16, 2)
```

5 batch train: 16, 16, 16, 16, 8.

Batch, step, epoch

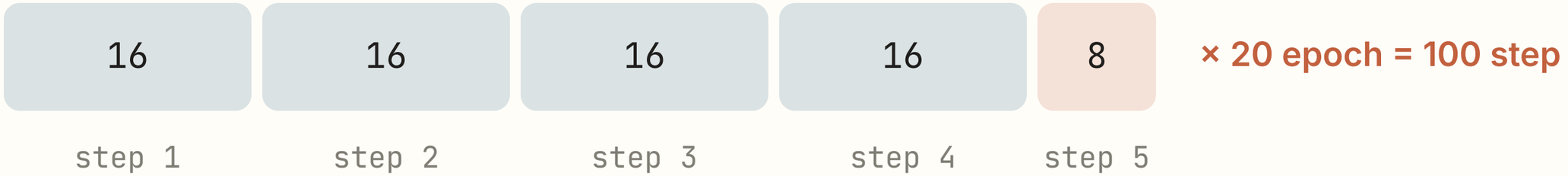
Batch: nhóm mẫu cùng tính gradient.

Step: một lần optimizer cập nhật.

Epoch: một lượt duyệt train.

72 mẫu, 5 step mỗi epoch

1 epoch = 72 mẫu train



batch cuối 8 mẫu vẫn là một lần cập nhật

72 mẫu train, batch size 16: mỗi epoch có 5 batch (bốn batch 16 và một batch 8), mỗi batch một step, nên 20 epoch là 100 step.

HỎI NHANH

Train có 72 mẫu, batch size 16, không bỏ batch cuối. Train 20 epoch thì optimizer cập nhật bao nhiêu lần?

A 20

B 80

C 100

D 1440

Vòng một batch



đổi thứ tự backward / zero_grad / step là mất hoặc dùng sai gradient

Mỗi batch đi đúng một vòng zero_grad → forward → loss → backward → step rồi mới sang batch tiếp theo.

Thứ tự có ý nghĩa

Đoạn trích của một batch: `xb` có shape `(B,2)`, model trả logits `(B,2)`; `yb` là nhãn lớp kiểu `long`, shape `(B,)`. `criterion` là cross-entropy.

```
optimizer.zero_grad(set_to_none=True)
logits = model(xb)
loss = criterion(logits, yb)
loss.backward()
optimizer.step()
```

`zero_grad` trước `backward`.

`step` sau `backward`.

Xóa gradient giữa hai bước ấy sẽ mất tín hiệu.

.grad cộng dồn

```
w = torch.zeros(2, requires_grad=True)
for xi in [torch.tensor([1., 2.]),
           torch.tensor([3., -1.])]:
    (xi @ w).backward()
    print(w.grad.tolist())
```

KẾT QUẢ

```
[1.0, 2.0]
[4.0, 1.0]
```

Lần hai: .

Thiếu zero_grad, bước cập nhật lớn dần.

HỎI NHANH

Một vòng lặp gọi `loss.backward()`, rồi `optimizer.zero_grad(set_to_none=True)`, rồi `optimizer.step()`. Train loss đứng yên ở mức ban đầu qua mọi epoch, và không có lỗi nào. Vì sao?

A Learning rate quá nhỏ nên mỗi bước gần như không đổi gì

B Gradient bị xóa trước `step()`, nên không có gì để cập nhật

C Gradient cộng dồn qua các batch nên các bước triệt tiêu nhau

D Backward chưa chạy xong vì `zero_grad` được gọi ngay sau nó

02

Đánh giá đúng trạng thái

Hai điều khiển độc lập

`model.eval()`

Đổi hành vi dropout/BatchNorm.

`with torch.no_grad():`

Ngừng ghi graph.

Validation thường cần cả hai.

HỎI NHANH

Model có lớp Dropout, tức lớp tắt ngẫu nhiên một số nơ-ron trong lúc train (bài 3.6 sẽ học kỹ). Hàm chấm validation có `with torch.no_grad():` nhưng quên `model.eval()`. Chuyện gì xảy ra?

A Dropout vẫn tắt ngẫu nhiên nơ-ron, nên điểm dao động

B `no_grad` tự chuyển model sang eval, nên vẫn đúng

C Graph vẫn được ghi lại, vì model còn ở chế độ train

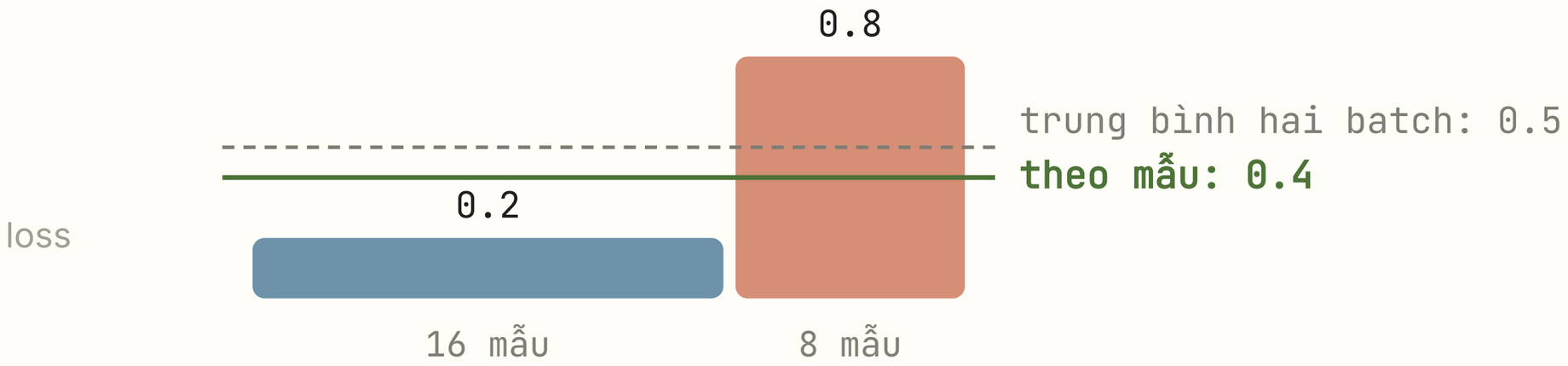
D Báo lỗi, vì Dropout cần gradient mới chạy được

Trung bình theo mẫu

Batch 16: loss 0.2 · batch 8: loss 0.8.

Không lấy .

Chiều rộng là số mẫu



Batch 16 mẫu loss 0.2 và batch 8 mẫu loss 0.8: chiều rộng là số mẫu; trung bình theo mẫu là 0.4, không phải 0.5.

HỎI NHANH

Một epoch có bốn batch 16 mẫu, batch nào cũng có loss trung bình 0.1, và một batch cuối 8 mẫu có loss trung bình 0.7. Loss trung bình theo mẫu của cả epoch là bao nhiêu?

A 0.10

B 0.17

C 0.22

D 0.40

Kết quả thật của loop

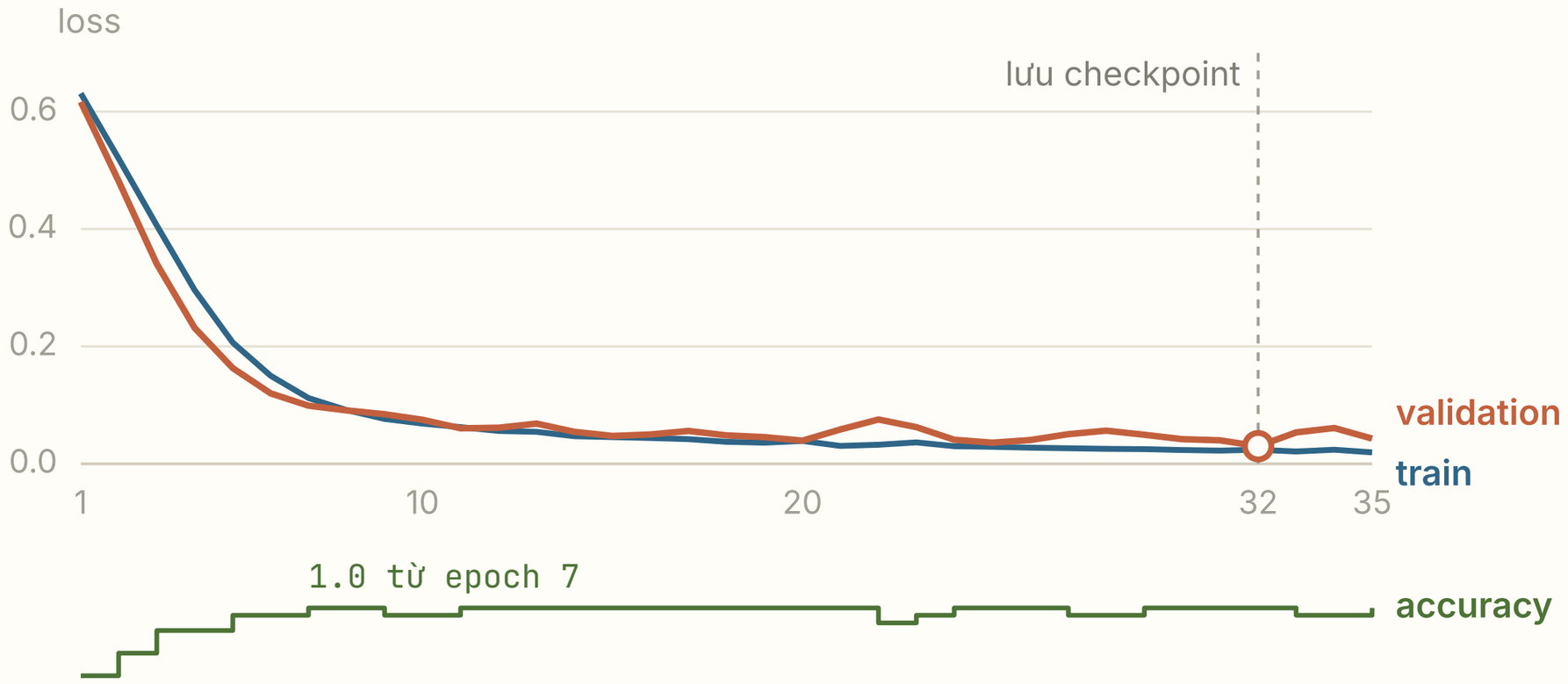
Epoch	Train loss	Val loss	Val acc
1	0.6312	0.6167	0.625
10	0.0687	0.0751	0.9583
20	0.039	0.0396	1.0
35	0.0197	0.0429	1.0

Train tốt thêm chưa bảo đảm validation tốt thêm.

Validation chỉ 24 mẫu: 21/35 epoch cùng accuracy 1.0, đầu tiên là epoch 7.

Loss phân biệt được, accuracy thì không.

35 epoch trên một hình



Log 35 epoch của bài: train loss giảm đều; validation loss thấp nhất ở epoch 32 rồi tăng nhẹ, nên checkpoint lấy epoch 32. Accuracy đạt 1.0 từ epoch 7 và không còn phân biệt được các epoch.

Cùng vòng train trên ảnh thật



bước 0 (epoch 0.0): accuracy validation 0.142, loss 2.313



bước 10 (epoch 0.1): accuracy validation 0.577, loss 1.658



bước 80 (epoch 0.9): accuracy validation 0.757, loss 0.719



bước 1880 (epoch 20.0): accuracy validation 0.852, loss 0.420

Cùng vòng train, chấm validation, lần này trên 6000 ảnh Fashion-MNIST thật: viền xanh là đoán đúng, viền cam là đoán sai.

Best không phải last

KẾT QUẢ

```
best epoch: 32
best validation: (0.03, 1.0)
```

Tiêu chí: validation loss nhỏ nhất.

Snapshot cần clone/deepcopy hoặc ghi tệp ngay.

Tensor trong `state_dict()` dùng chung bộ nhớ với tham số.

HỎI NHANH

Code chọn checkpoint ghi `best_state = model.state_dict()`, không clone, mỗi khi validation loss tốt hơn. Train hết 35 epoch rồi `model.load_state_dict(best_state)`. Model có trọng số của epoch nào?

A Epoch 32, epoch có validation loss nhỏ nhất

B Epoch 1, lần đầu điều kiện được thỏa

C Không xác định, vì tensor cũ đã bị giải phóng

D Epoch 35, vì `best_state` thay đổi theo model

HỎI NHANH

Giả sử vòng lặp trên chọn checkpoint theo validation accuracy, với điều kiện `valid_accuracy > best_accuracy` và `best_accuracy` khởi tạo bằng 0, thay vì theo loss. Checkpoint là epoch nào?

A Epoch 7, lần đầu accuracy đạt 1.0

B Epoch 20, mốc log đầu tiên có accuracy 1.0

C Epoch 32, cũng là epoch có loss nhỏ nhất

D Epoch 35, lần cuối accuracy đạt 1.0

03

Khôi phục và kiểm lại

Gói lưu cho suy luận

- `state_dict` của model.
- Mean/std của train.
- Code dựng kiến trúc.
- Thứ tự feature, bảng chỉ số → tên lớp.

Muốn train tiếp còn cần state của optimizer, scheduler và RNG.

Chỉ tải checkpoint tin cậy; packet của bài dùng `weights_only=True`.

HỎI NHANH

Phiên Colab bị ngắt ở epoch 20, và checkpoint chỉ có `state_dict` của model. Ta tạo một Adam mới rồi train tiếp 15 epoch. So với chạy liền 35 epoch thì sao?

A Giống hệt, vì trọng số model đã mô tả đủ trạng thái

B Giống hệt, nếu đặt lại seed như lúc đầu

C Khác, vì các trung bình động của Adam và thứ tự xáo dữ liệu đều bắt đầu lại

D Báo lỗi, vì `load_state_dict` đòi cả state của optimizer

Model mới, cùng logits

```
fresh = make_model()
fresh.load_state_dict(packet['state'])
fresh.eval()
```

KẾT QUẢ

```
same logits: True
classes: [1, 0]
```

Đầu vào thô được chuẩn hóa bằng mean/std đã lưu.

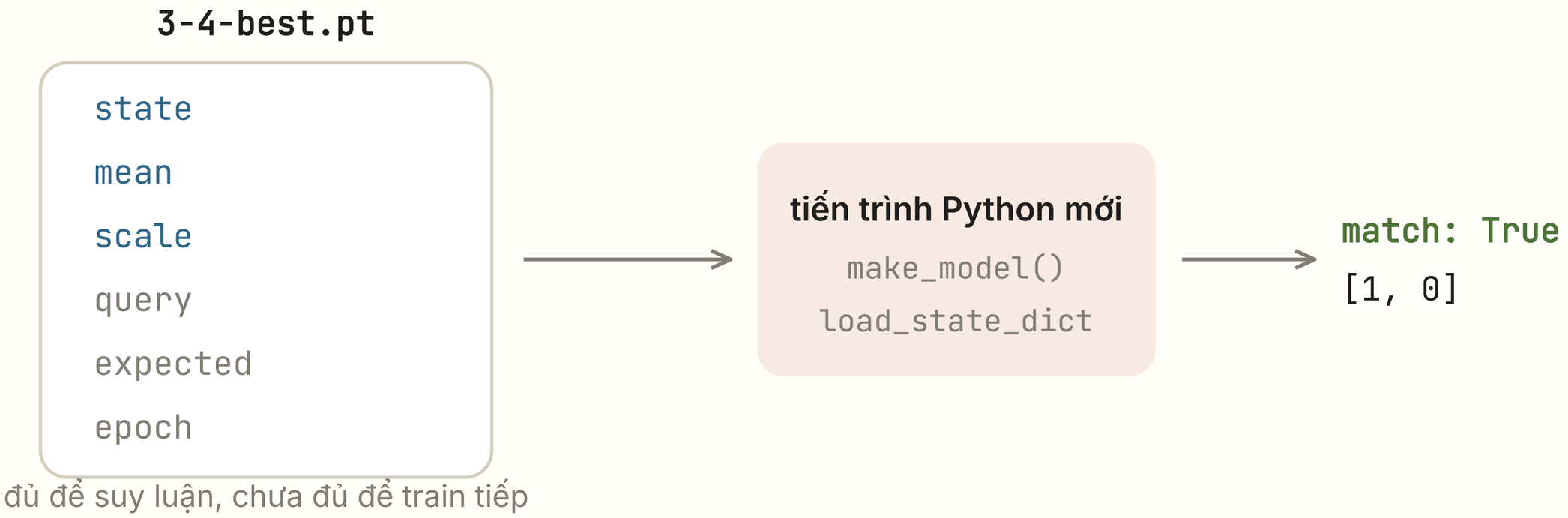
Một tiến trình Python mới

KẾT QUẢ

```
new process match: True
new process classes: [1, 0]
```

Không dùng biến còn sót trong notebook.

Packet tự đủ



Packet checkpoint của bài: trọng số, mean và scale của train, cùng đầu vào kiểm và logits mong đợi; một tiến trình Python mới chỉ dùng packet mà vẫn cho cùng kết quả.

HỎI NHANH

Trong notebook, model mới nạp checkpoint đã cho `same logits: True`. Vì sao bài vẫn chạy thêm một script ở tiến trình Python mới?

A `torch.load` chỉ đọc đúng tệp khi chạy ở tiến trình khác

B Tiến trình mới đổi seed nên phép so khắt khe hơn

C `allclose` trong cùng tiến trình luôn trả về `True`

D Notebook có thể đang dựa vào biến cũ không có trong packet

Tái lập cần gì?

Seed model, split và loader.

Ghi phiên bản, cấu hình, checkpoint.

Seed không hứa giống từng bit trên mọi phần cứng.

So hai cấu hình qua vài seed, không qua một lần chạy.

HỎI NHANH

Thử learning rate 0.03 và 0.01, mỗi cấu hình chạy một lần với cùng seed 21. Validation loss tốt nhất lần lượt là 0.030 và 0.028. Kết luận nào hợp lý?

A Chọn 0.01, vì cùng seed nên chênh lệch chỉ do learning rate

B Chọn 0.03, vì chênh 0.002 quá nhỏ nên giữ cấu hình cũ

C Chưa chọn được, cần chạy thêm vài seed rồi so trung bình

D Chọn 0.01, vì validation loss thấp hơn là đủ để quyết định

Khi đi thi

- Chạy thử vài batch, đo thời gian một epoch.
- Lưu best ra tệp ngay, đúng metric và hướng so.
- Checkpoint kèm scaler, thứ tự feature, bảng lớp.
- Restart, chạy đúng script nộp, kiểm ID và số hàng.
- Không chọn epoch bằng test hay bảng công khai.

04

Luyện đề

LUYỆN ĐỀ 1

Một vòng lặp quên hẳn `optimizer.zero_grad()`. Tham số bắt đầu bằng 1, optimizer là SGD với learning rate 0.1, và ở batch nào gradient của loss theo cũng bằng 2. Sau 3 batch, và `w.grad` bằng bao nhiêu?

A , `w.grad` bằng 2

B , `w.grad` bằng 6

C , `w.grad` bằng 2

D , `w.grad` bằng 6

PHỎNG THEO NOAI 2026 Singapore · Vòng sơ loại · Câu 126

LUYỆN ĐỀ 2

Lưu MLP của bài, gồm `Linear(2, 8)`, `ReLU`, `Linear(8, 2)`, bằng `torch.save(model.state_dict(), 'best.pt')`. Tệp này chứa gì?

A 4 tensor, tổng cộng 42 số: weight và bias của hai lớp Linear

B 6 tensor, thêm một mục cho ReLU và một mục cho kiến trúc

C 4 tensor trọng số kèm `.grad` để train tiếp được

D Cả object model, nạp lại không cần dựng kiến trúc

PHỎNG THEO NOAI 2026 Singapore · Vòng sơ loại · Câu 134

LUYỆN ĐỀ 3

Phân loại 10 lớp cân bằng, validation accuracy trong notebook là 0.91. Script nộp chạy ở phiên mới, nạp đúng checkpoint, chuẩn hóa bằng mean và std đã lưu, số hàng và thứ tự ID khớp file mẫu. Vậy mà điểm chấm chỉ khoảng 0.10. Giả thuyết nào khớp nhất?

A Model overfit validation, cần thêm regularization rồi train lại

B Bảng đổi chỉ số ra tên lớp ở script nộp khác với lúc train

C Test lệch phân phối nên scaler cần fit lại trên test

D Phiên mới có seed khác nên dự đoán thành ngẫu nhiên

PHỎNG THEO NOAI 2026 Singapore · Vòng sơ loại · Câu 284

Từ “học được” tới “dùng được”

Loop đúng · validation đúng · snapshot thật · suy luận mới.

Làm **10 câu kiểm tra cuối bài** trên trang bài học.

Trong notebook, khởi động lại runtime rồi chạy riêng phần nạp checkpoint: logits phải khớp.